

CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Course website
(GitHub) is down.
Access slides on
[EdStem](#) / QR code

RISC-V Basics



Special Numbers, Overflow, and Underflow

- [Floats] Special Numbers, Overflow and Underflow
- The RISC-V Instruction Set Architecture
- Elements of Architecture: Registers
- Add/Sub Instructions
- Immediates, Register Zero



(change of gears)

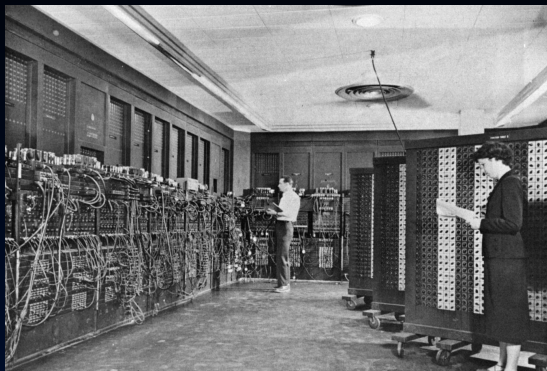
<https://www.wikihow.com/Shift-a-Manual-Transmission>



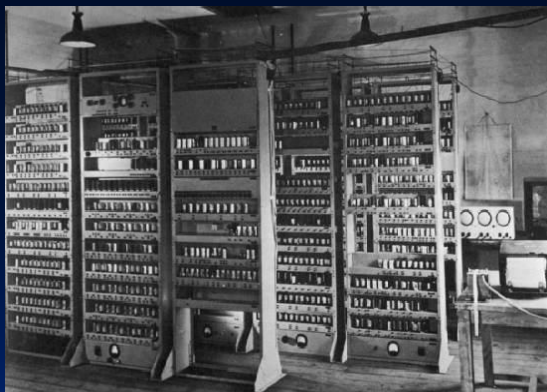
The RISC-V Instruction Set Architecture

- [Floats] Special Numbers, Overflow and Underflow
- The RISC-V Instruction Set Architecture
- Elements of Architecture: Registers
- Add/Sub Instructions
- Immediates, Register Zero

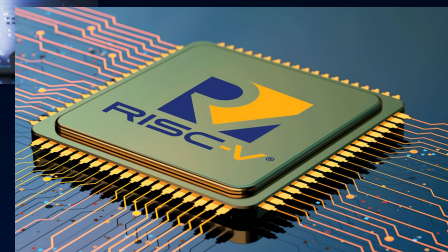
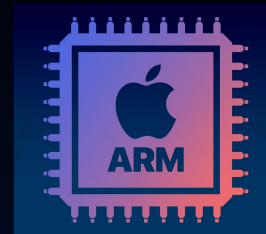
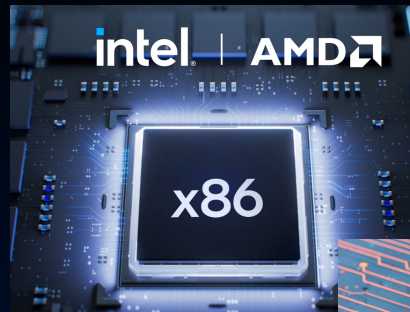
From Circuits to Programmable Circuits



ENIAC (1946):
physical rewiring;
highly restrictive



EDSAC (1949):
First General
Stored-Program
Computer



Basic job of a **CPU**:
execute lots of
instructions.

Great Idea #1: Abstraction (Levels of Representation/Interpretation)



High Level Language
Program (e.g., C)

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

Compiler

Assembly Language
Program (e.g., RISC-V)

```
lw    x3, 0(x10)
lw    x4, 4(x10)
sw    x4, 0(x10)
sw    x3, 4(x10)
```

Assembler

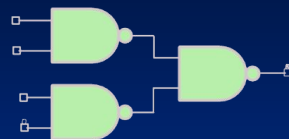
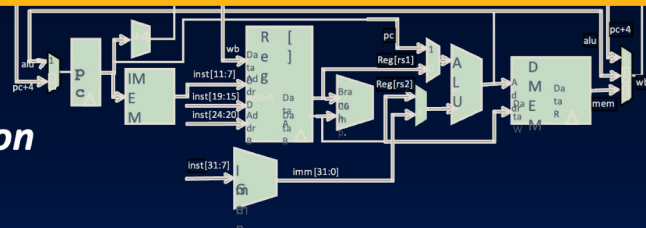
Machine Language
Program (RISC-V)

```
1000 1101 1110 0010 0000 0000 0000 0000
1000 1110 0001 0000 0000 0000 0000 0100
1010 1110 0001 0010 0000 0000 0000 0000
1010 1101 1110 0010 0000 0000 0000 0100
```

Hardware Architecture Description
(e.g., block diagrams)

Architecture Implementation

Logic Circuit Description
(Circuit Schematic Diagrams)

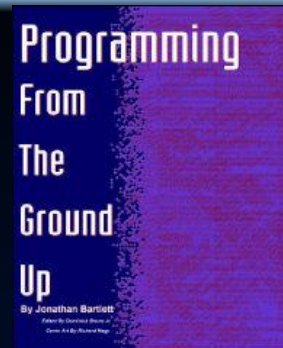




Book: Programming From the Ground Up

"...I've found that the key difference between mediocre and excellent programmers is whether or not they know assembly language. **Those that do tend to understand computers themselves at a much deeper level.** Although [almost!] unheard of today, this concept isn't really all that new -- there used to not be much choice in years past. Apple computers came with only BASIC and assembly language, and there were books available on assembly language for kids. This is why the old-timers are often viewed as 'wizards': they had to know assembly language programming."

-- slashdot.org comment, 2004-02-05



ISAs: RISC vs. CISC

- Different CPUs implement different **Instruction Set Architectures** (ISAs):
 - ARM (cell phones), Intel x86 (i9, i7, i5, i3), IBM/Motorola PowerPC (old Macs), MIPS, **RISC-V**, ...
 - Early trend was to add more and more instructions to new CPUs to do elaborate operations. (e.g., Intel x86)
- **Reduced Instruction Set Computer (RISC)** philosophy:
 - Cocke IBM, **Patterson**, **Hennessy**, 1980s.
 - Keep the instruction set small and simple. Write complicated software by composing simpler operations.
 - **Makes it easier to build fast hardware!**
 - At the time, this went against the conventional wisdom... (he who laughs last, laughs best)

Patterson and Hennessy win Turing Award! (2017)



<https://engineering.berkeley.edu/news/2018/06/patterson-wins-turing-award/>



RISC-V Architecture

- New (2015) open-source, license-free ISA spec
 - Supported by growing shared software ecosystem
 - Appropriate for all levels of computing system, from microcontrollers to supercomputers
- Why **RISC-V** instead of **Intel x86-64**?
 - RISC-V is simple and elegant. We don't need to get bogged down in gritty details.
 - It is open-source, so we can use it to design CPUs.
 - **Go Bears!**

The image shows a detailed technical document titled 'RISC-V Green Card' which serves as a reference for the RISC-V instruction set. It includes sections for 'Reference Data', 'ARCHITECTURE CORE INSTRUCTION SET', and 'CORE INSTRUCTION FORMATS'. The document is organized into columns with various instruction codes and their corresponding operations. It also includes a table for 'CORE INSTRUCTION FORMATS' showing the bit fields for different instruction types. The document is a comprehensive reference for anyone working with RISC-V architecture.

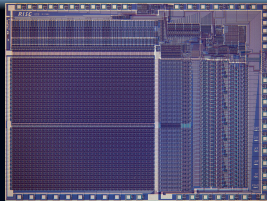
RISC-V Green Card

Yan, SP25

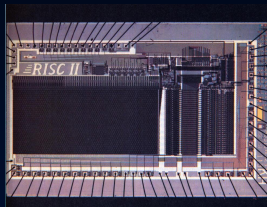
RISC-V Origins

[source](#)

- Started in Summer 2010 to support open research and teaching at UC Berkeley
 - Lineage can be traced to RISC-I/II projects (1980s)
- As the project matured, it migrated to RISC-V foundation (www.risc-v.org)
- Many commercial and research projects based on RISC-V.
 - Open-source and proprietary
 - Widely used in education



RISC-I



RISC-II



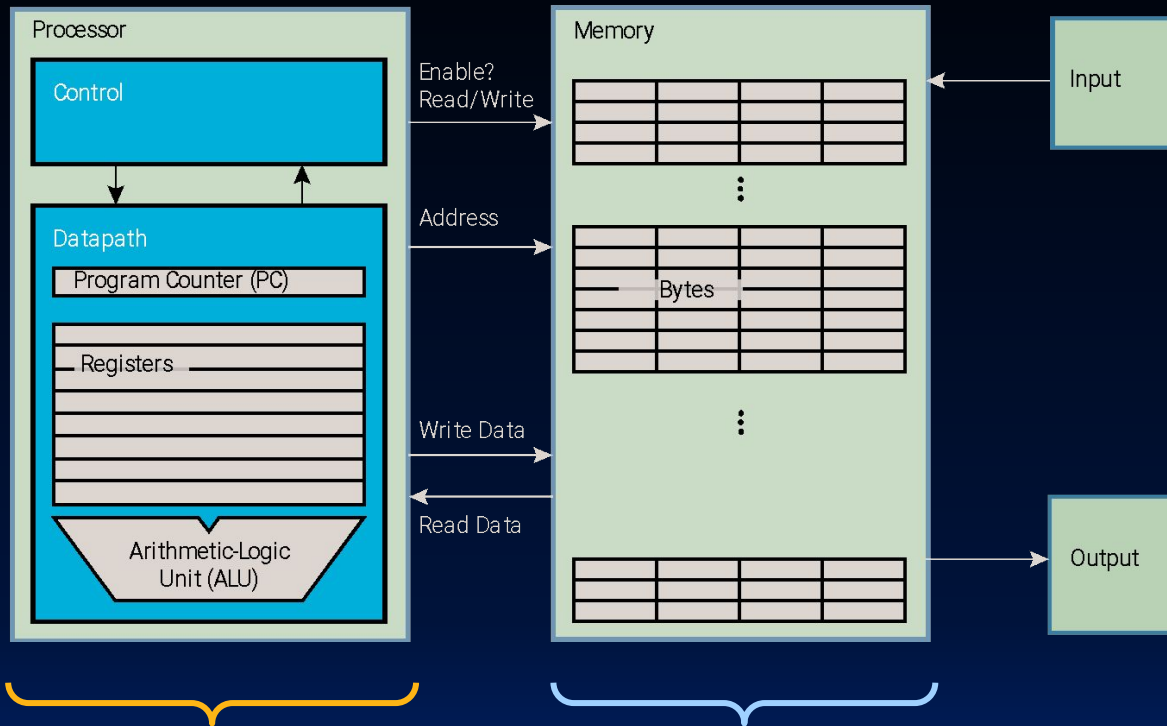


Elements of Architecture: Registers

- [Floats] Special Numbers, Overflow and Underflow
- The RISC-V Instruction Set Architecture
- Elements of Architecture: Registers
- Add/Sub Instructions
- Immediates, Register Zero

Computer Layout

- Two main parts:
 - CPU/Processor**, responsible for computing
 - Main memory**, responsible for long-term data storage
- CPU is designed to be **fast**. (why?)
 - CPU therefore stores a small amount of data through **registers**.



CPU: Compute

09-RISC-V Basics (13)

Memory: Load/Store



ISAs (1/2): Architecture

- An **Instruction Set Architecture (ISA)** defines what operations a particular computer supports, and how it implements them.
- An ISA like RISC-V generally defines three components:
 - a. Fundamental architectural features and design



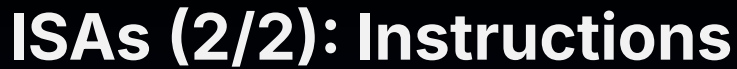
RV32I: Register Specification

- Registers are **inside** the processor, i.e., built into hardware
 - ISA determines number of registers AND size of registers.
- A compiler follows the ISA and maps **C variables to RISC-V registers** across declarations, function calls, etc.
 - RISC-V code must be very carefully put together to efficiently use registers...!

RV32I: RISC-V 32-bit Integer ISA

- There are 32 registers.
"Goldilocks" principle
- Registers are word-sized, i.e., 32-bits

Register Names and Numbers



- add** x18 x19 x20 ➡ 00000000101010011000100100110011
machine code
- operation name** **register operands**



Ok, enough already...gimme my RV32!!!!

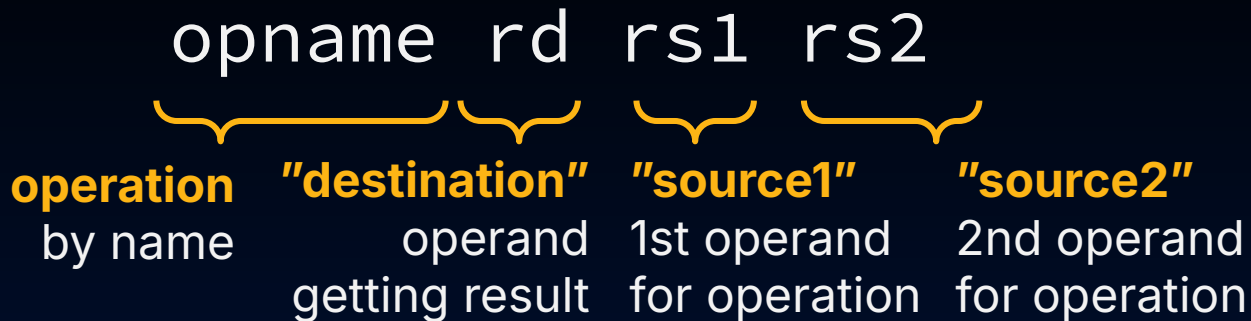


Add/Sub Instructions

- [Floats] Special Numbers, Overflow and Underflow
- The RISC-V Instruction Set Architecture
- Elements of Architecture: Registers
- Add/Sub Instructions
- Immediates, Register Zero



RISC-V Arithmetic Instruction Syntax



- Syntax is rigid:
 - 1 operator, 3 operands.
 - Why? Keep hardware simple via regularity.



RISC-V Addition, Subtraction Instructions

opname	rd	rs1	rs2
operation by name	"destination" operand getting result	"source1" 1st operand for operation	"source2" 2nd operand for operation

Addition

RISC-V `add x1 x2 x3`

C `a = b + c;`

(int a stored in register x1, and
so on)

Subtraction

RISC-V `sub x4 x5 x6`

C `d = e - f;`

(order of operands matter in sub!)



RISC-V Arithmetic, Example 1

- Translate the following C statement:

```
a = b + c + d - e;  
x10 x1  x2  x3  x4
```

C variables
RISC-V registers

- Solution: Break into multiple instructions!

```
add x10  x1 x2      # a_temp = b + c  
add x10 x10 x3      # a_temp = a_temp + d  
sub x10 x10 x4      # a = a_temp - e
```

A single statement in C may translate to multiple instructions in RISC-V.

RISC-V comments (in-line)

- Margaret Hamilton, Director of Software Engineering, Division of MIT Instrumentation Laboratory
 - Built Apollo Guidance Computer
 - Command Module (navigation, lunar landing)
 - 72KB of memory on board Eagle (Apollo 11, 1969)



Margaret Hamilton and Apollo code
Wikimedia Commons

Presidential Medal
of
Freedom
(2016)



179	TC	BANKCALL	# TEMPORARY, I HOPE HOPE HOPE
180	CADR	STOPRATE	# TEMPORARY, I HOPE HOPE HOPE
181	TC	DOWNFLAG	# PERMIT X-AXIS OVERRIDE

245	CAF	CODE500	# ASTRONAUT: PLEASE CRANK THE
246	TC	BANKCALL	# SILLY THING AROUND
247	CADR	GOPERF1	
248	TCF	GOTOP00H	# TERMINATE
249	TCF	P63SPOT3	# PROCEED SEE IF HE'S LYING

250				
251	P63SPOT4	TC	BAN	39 ## It traces back to 1965 and the Los Angeles riots, and was inspired
252		CADR	SET	40 ## by disc jockey extraordinaire and radio station owner Magnificent Montague.
253				41 ## Magnificent Montague used the phrase "Burn, baby! BURN!" when spinning the
254		TC	POS	42 ## hottest new records. Magnificent Montague was the charismatic voice of
255		CADR	BUR	43 ## soul music in Chicago, New York, and Los Angeles from the mid-1950s to
				44 ## the mid-1960s.



RISC-V Arithmetic, Example 2

- Translate the following C statement:

`f = (g + h) - (i + j);`

C variables

`x19` `x20` `x21` `x22` `x23`

RISC-V registers

A. `add` `x5` `x20` `x21`
 `add` `x6` `x22` `x23`
 `sub` `x19` `x5` `x6`

B. `add` `x19` `x20` `x21`
 `sub` `x19` `x19` `x22`
 `sub` `x19` `x19` `x23`



Yan, SP25

L08: Which of the following is a valid translation of the C statement?

Translate the following C statement:

```
f = (g + h) - (i + j);
```

x19 x20 x21 x22 x23

- Assume the above mapping for C variables $\leftrightarrow$ RISC-V registers.

A.

```
add x5, x20, x21
add x6, x22, x23
sub x19, x5, x6
```

B.

```
add x19, x20, x21
sub x19, x19, x22
sub x19, x19, x23
```

A

0%

B

0%

Both of the above

0%

Something else

0%





RISC-V Arithmetic, Example 2

- Translate the following C statement:

`f = (g + h) - (i + j);`

`x19` `x20` `x21` `x22` `x23`

C variables

RISC-V registers

- Solution A: Use **temporary registers**.
 - (any existing data in `x5`, `x6` will be overwritten)
- Solution B: Have a **clever compiler** that avoids any temporary registers...!

`a_temp: x5, b_temp: x6`

A. `add x5 x20 x21`
`add x6 x22 x23`
`sub x19 x5 x6`

`f = g + h - i - j;`

B. `add x19 x20 x21`
`sub x19 x19 x22`
`sub x19 x19 x23`

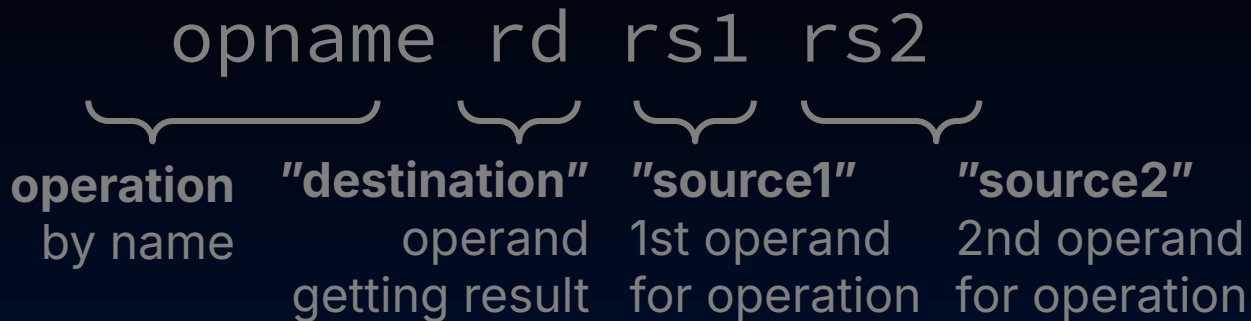


Immediates, Register Zero

- [Floats] Special Numbers, Overflow and Underflow
- The RISC-V Instruction Set Architecture
- Elements of Architecture: Registers
- Add/Sub Instructions
- Immediates, Register Zero



RISC-V Addition, Subtraction Instructions



Addition

RISC-V `add x1 x2 x3`

C `a = b + c;`

(int a stored in register x1, and
so on)

Subtraction

RISC-V `sub x4 x5 x6`

C `d = e - f;`

(order of operands matter in sub!)

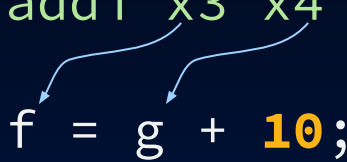


Immediates

- **Immediates** are numerical constants in assembly instructions.
 - Immediate bits are **directly** encoded into instructions
 - So ISA defines separate instructions for immediates

Add Immediate

RISC-V	<code>addi x3 x4 10</code>
C	<code>f = g + 10;</code>





Subtract Immediate?

- **Immediates** are numerical constants in assembly instructions.
 - Immediate bits are **directly** encoded into instructions
 - So ISA defines separate instructions for immediates

Add Immediate

RISC-V `addi x3 x4 10`

C `f = g + 10;`

Subtract Immediate?

RISC-V `addi x3 x4 -10`

C `f = g - 10;`

RISC Philosophy: Reduced.

If an operation can be decomposed into **equivalent/simpler operations**, don't include it in the ISA.

Register Zero

- One particular immediate, the number zero (**0**), appears very often in code.
- RISC-V **hardwires** the register zero (**x0**) to value **0**.

```
addi x3 x0 0xff
```

```
sub x3 x0 x4
```

```
add x0 x3 x4
```

```
      f      g  
x3    x4
```

What are equivalent C statements, if any?



Yan, SP25



Register Zero

- One particular immediate, the number zero (**0**), appears very often in code.
- RISC-V **hardwires** the register zero (**x0**) to value **0**.

f g
x3 x4

```
addi x3 x0 0xff            # f = 0xff;
```

```
sub  x3 x0 x4              # f = -g;
```

```
add  x0 x3 x4              # // no-op ("no operation")
```

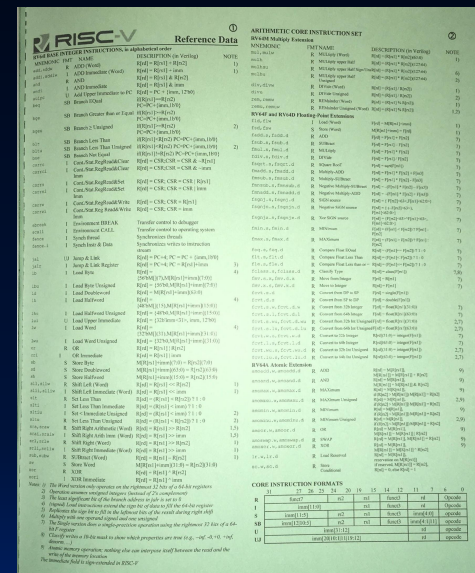


See you next time!



RISC-V Resources for CS61C

- CS 61C Reference Card
 - <https://cs61c.org/sp25/pdfs/resources/reference-card.pdf>
 - Lists out the entire base architecture
- Venus
 - <https://venus.cs61c.org/>
 - Online RISC-V simulator
- Full Technical Specifications
 - <https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications>
 - → Volume I: unprivileged ISA



RISC-V Green Card

Yan, SP25



Backup Slides

- [Floats] Special Numbers, Overflow and Underflow
- The RISC-V Instruction Set Architecture
- Elements of Architecture: Registers
- Add/Sub Instructions
- Immediates, Register Zero

HLL vs. Assembly: Data representation

C, Java

- In C (and most high-level languages), variables declared first and given a **type**.
 - Each variable can ONLY represent a value of the type it was declared as (cannot mix/match int/char variables).
- In high-level languages, **variable types determine operation**.

```
int *p = ...;  
p = p + 2;  
int x = 42;  
x = 3 * x;
```

RISC-V

- Assembly operands are registers.
 - Registers are limited number of special locations, built directly into the hardware.
 - RISC-V: Operations can only be performed on register operands.
- In assembly language, the registers have no type.
 - Register contents are just bits.
- **Operation** determines "type," i.e., how register contents are treated.
 - as value, memory address, etc.

HLL vs. Assembly: Operations

C, Java

- Each line can contain multiple operations.
`a = b * 2 - (arr[2] + *p);`
- Common operations:
 - `=, +, -, *, /, ...`
 - Here, `*` includes both multiply and dereference

RISC-V

- Each statement is an **instruction**.
 - An instruction executes exactly one short list of simple commands.
- Each line of assembly code contains at most 1 instruction.
`add x1 x2 x3`
`sub x4 x5 x6`

The compiler **translates** from higher-level language down to assembly...
...so RISC-V instructions are often **closely related** to common C/Java operations.

Concept Check: True or False?

1. **Types** are associated with **declaration in C** (normally), but are associated with **instructions** (operators) in RISC-V.
2. Since there are only **32 registers**, we can't write RISC-V for C expressions that contain > 32 variables.
3. If **p** (stored in x9) were a pointer to an array of ints, then **p++;** would be `addi x9, x9, 1`.

- | | |
|----|-----|
| | 123 |
| A. | FFF |
| B. | FFT |
| C. | FTF |
| D. | FTT |
| E. | TFF |
| F. | TFT |
| G. | TTF |
| H. | TTT |

Concept Check: True or False?

1. **Types** are associated with **declaration in C** (normally), but are associated with **instructions** (operators) in RISC-V.
2. Since there are only **32 registers**, we can't write RISC-V for C expressions that contain > 32 variables.
3. If **p** (stored in x9) were a pointer to an array of ints, then **p++;** would be `addi x9, x9, 1`.

1. **True**. we saw that on an earlier slide.
2. **False**. we saw how to break up longer equations to smaller ones already.
3. **False**. Don't forget that ints are (usually) 4 bytes wide, so instruction would be `addi x9, x9, 4`.

	123
A.	FFF
B.	FFT
C.	FTF
D.	FTT
E.	TFF
F.	TFT
G.	TTF
H.	TTT